Assignment 1 Report

SIT215 – Computational Intelligence

Trimester 1, 2026

Student Name: Tien Huy Nguyen
Student ID: s223009962

Tasks Completed	Completed (✓)	Notes (Optional)
Task 1: Environment modelling and problem formulation	☑	
Task 2: Search-based agent implementation and comparison • A^* • Selected uninformed algorithm	☑	
Task 3: Success metrics and evaluation	☑	
Task 4: New heuristic design and performance	☑	

1. Problem Formulation

1.1 Problem Context

HanoiFastDelivery is a courier and parcel delivery company seeking to enhance its competitiveness in Vietnam's growing logistics industry. With the continuous expansion of e-commerce and urban delivery demand, the company is interested in developing an intelligent route planning system to improve delivery efficiency and service reliability. This is particularly relevant in Hanoi, where dense traffic, one-way roads, and diverse road conditions pose major challenges to daily delivery operations. The parcel delivery market is highly competitive, with established companies such as Viettel Post, Vietnam Post, and J&T Express dominating the sector.

Before expanding its service more widely across Hanoi and potentially to other cities, **HanoiFastDelivery** requires an effective route planning solution for urban delivery. Therefore, this report focuses on a selected area of Hanoi to examine the feasibility and competitiveness of the proposed approach before scaling it to the whole city. The map is given below.

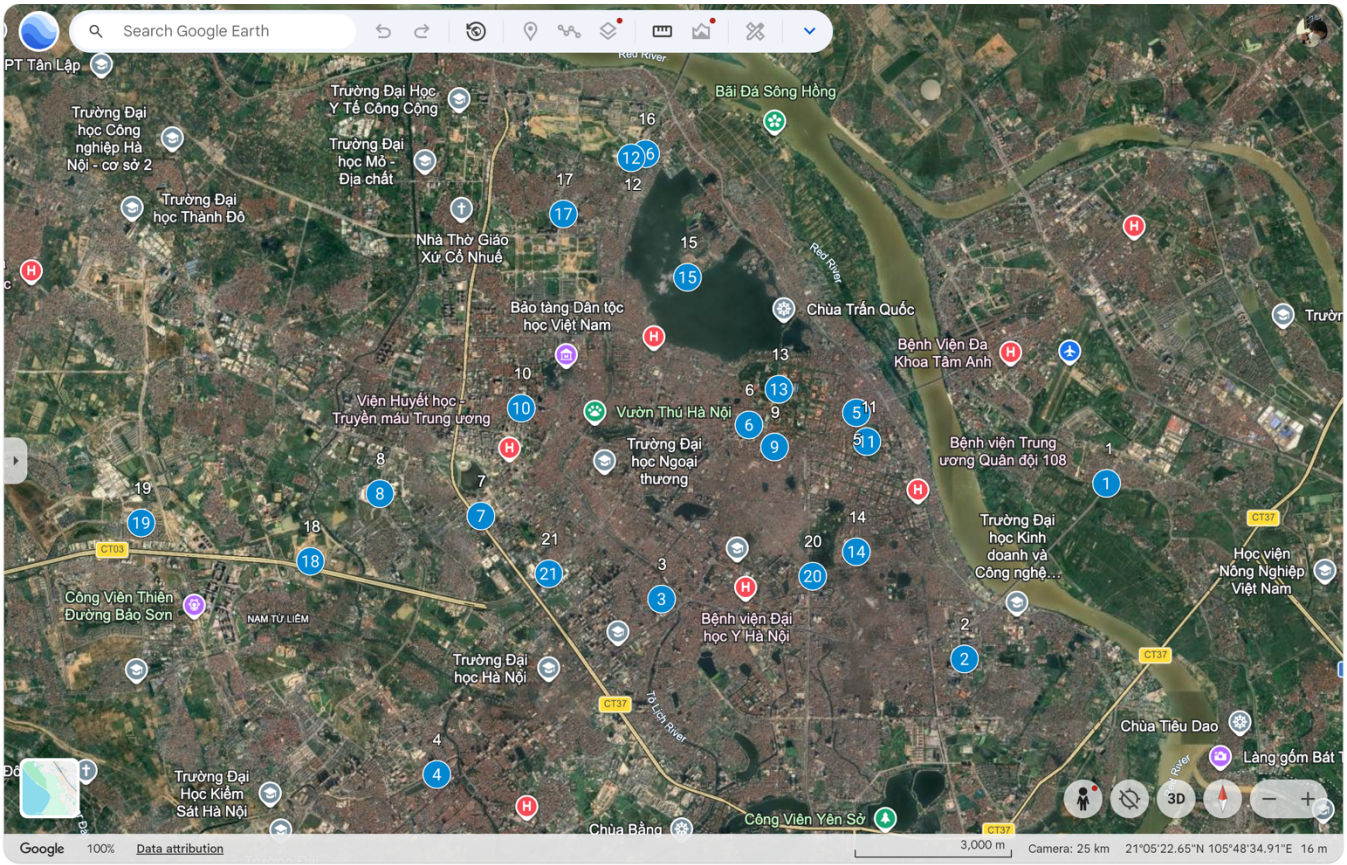

Figure 1: A snapshot of a selected region of Hanoi Capital with 21 landmarks adapted from Google Maps.

1.2 Problem Constraints and Environment Selection Reasoning

The chosen area in Hanoi is a crowded urban centre, as the map indicates, which puts a huge strain on the transport infrastructure, leading to a high traffic load and the appearance of traffic jams and poor road conditions. Express delivery firms should thus implement appropriate strategies to distribute transportation and management resources effectively and, at the same time, avoid handling delicate packages and deliver goods in a timely manner. Moreover, the planners must consider vehicle restrictions of certain roads, traffic during various times of day, one-way roads, and small road widths. In this context, the route feasibility is

determined by two dynamic constraints, i.e., vehicle constraints and traffic flow as well as several static constraints based on the roads, e.g., distance and direction as well as road condition.

The 21 most significant landmarks chosen, as indicated above, are scattered around the Hanoi map, and some are close to major roads and others within smaller internal streets. Moreover, there is a wide range of distances between landmarks. There is a weight that is allocated to each path segment, which is the estimated transportation cost using traffic flow and distance. It is more realistic than a distance-alone approach, as the real journey time on a road can change because of road conditions. This model does not consider toll charges since most of the toll stations are in the outskirts of Hanoi.

2. Modelling Approach

2.1 PEAS Framework

The problem statement above can be summarised more concisely by the utilisation of PEAS framework to formally define the rationality standards of truck delivery agent.

PEAS Component	Description for HanoiFastDelivery
Performance Measures	+ Fuel cost optimisation. + Correct time and parcel delivery + Protection of fragile items during transportation.
Environment	+ Multi-agent. + Dynamic sequential. + Route restrictions and traffic conditions. + Partially observable.
Actuators	+ Steering. + Acceleration. + Brake
Sensors	+ GPS.

	+ Clock. + Delivery status. + Traffic observations.

2.2 Environment Abstraction into State Space: A relaxed version

Having studied the PEAS framework above, it is apparent that the delivery route planning problem in Hanoi is far from simple. Although the original problem reflects real-world delivery conditions well, it is difficult to implement an effective and efficient intelligent agent in such a complex environment. Therefore, in this project, the problem is relaxed by preserving the most important static constraints while simplifying dynamic constraints into fixed modelling factors. In addition, to better capture the varying transportation costs between locations, the environment is represented as a **weighted graph**, in which different weights are assigned to the paths connecting pairs of landmarks.

Unlike the real Hanoi road network, which contains many roads ranging from major avenues to narrow internal streets, the relaxed problem only includes commonly used and practically relevant roads between the selected destinations. As a result, a graph, which is a discrete data structure, is used to represent the environment because it allows the states of the problem to be defined more clearly and makes search implementation more manageable.

To preserve the static constraints of the original problem, especially road direction, a **directed graph** is used where necessary, allowing the agent to move only in valid directions. This is important because some roads in Hanoi are one-way, while others permit movement in both directions. To maintain route feasibility, the graph should be sufficiently connected so that meaningful paths can exist between landmarks. Another important static constraint is **road condition**, since road quality may vary across the city. Poorly maintained roads may increase the chance of parcel damage, especially for fragile items. To model this factor, a **risk coefficient**, denoted as $\tau\ (u, v) \in (0, 1)$ is added to reflect the probability that items being transported can be destroyed along the route between node u to node v. The last static constraint is the distance between two landmarks, which is represented by $d\ (u, v)$ which can be approximated by map services (e.g., Google Maps).

The relaxed problem is almost a copy of the static constraints; however, it also tries to capture certain dynamic features of the real environment, especially **traffic flow**. This aspect has a powerful influence on the travel time and can also lead to the possibility of delays or transport-related problems. Hence, there is a **coefficient** of **traffic congestion** represented as $\psi(u, v)$, is presented to indicate the degree of congestion on a road segment. Also, the **estimated time to travel** denoted as $t\ (u, v)$is also presumed to be an important factor and can also be obtained with Google Maps. Limited access to roads at specific times is another dynamic factor in the real delivery operations. This is, however, hard to model in a simplified graph representation and is not, therefore, included in the present implementation. Rather, the model is aimed at modelling one-way and two-way road movement, as well as distance, risk, and congestion.

In general, this graph abstraction retains the key properties of the Hanoi delivery problem and renders it computationally feasible. It offers a well-defined state-space model where landmarks become nodes and road connections weighted edges, enabling search methods like **A*** to be efficiently used.

2.2.1 Cost Model

In this section, we present the mathematical model that is used to reasonably assign the costs to different path segments connecting the landmarks. This cost model incorporates several functions that strictly follows the interpretations above. Remarkably, the information of this model is not given to the agent by any means.

Given a graph $G = (V, E$, for an edge $(u, v) \in E$ The corresponding weight, denoted as $\Omega\ (u, v)$, given by:

$$\omega\ (u, v) = \alpha\ .\ d\ (u, v) + \beta . t\ (u, v) \quad (1)$$

$$\Omega\ (u, v) = \left(1 + \tau\ (u, v) + \psi(u, v)\right) . \omega\ (u, v) \quad (2)$$

where:

- $d\ (u, v)$: estimated distance (in km) adapted from Google Maps
- α: fuel comsumption cost, in our context, $\alpha = 2.3$.
- $t\ (u, v)$: estimated time (in hour) adapted from GoogleMaps
- β: time contribution coefficient, in our context, $\beta = 2 * \alpha = 4.6$.
- $\tau\ (u, v) \in (0, 1)$: risk factor
- $\psi\ (u, v) \in (0, 1)$: traffic congestion coefficient

Beyond the aforementioned factors contributing to the cost connecting two landmarks, we introduce two auxiliary coefficients, i.e. $\beta\ and\ \alpha$, which demonstrates our focus when searching for an optimal path, which are fuel consumption and delivery time. Noticeably, we let $\beta > \alpha$ as we would like to emphasize the **timely delivery** mission of **HanoiFastDelivery**. In other words, we may prioritise routes that are a little further but fewer vehicles, and thereby move faster, over those are less distant but overly crowded.

Briefly speaking, this cost model reflects our concerns when finding the optimal path, by putting into consideration several factors and aggregating them by addition and multiplication. Firstly, $\omega\ (u, v)$ considers time and distance, which are two important factors in our problem, and allocates different coefficients, $\alpha\ and\ \beta$ to specifies our priority. Secondly, we amplify this path cost $\omega\ (u, v)$by considering traffic congestion degree $\psi(u, v)$ and the road conditions $\tau\ (u, v)$. Lastly, we add up these amplified costs with our pure one, to arrive at the final weight, i.e. $\Omega\ (u, v)$.

2.2.2 State Space Representation

The below graph represents the relaxed problem, which includes nodes, numbered from 1 to 21, akin to those in Figure 1. Additionally, the weights are assigned to edges connecting pairs of nodes, which were computed accordingly to the proposed **Cost Model** above. Noticeably, the weights were rounded for the ease of visualisation.

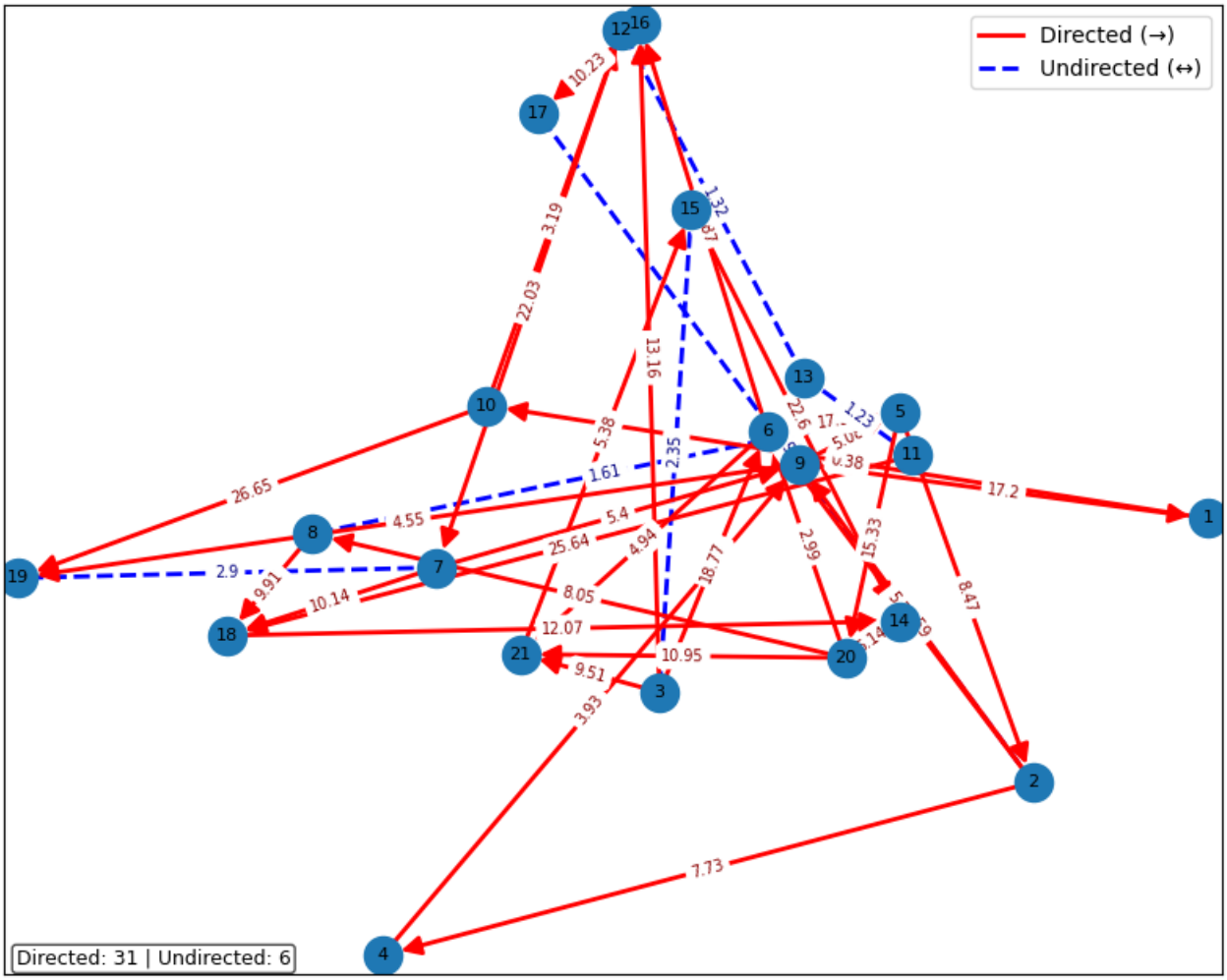

Figure 2: A state space representation of the relaxed problem

3. Search Strategies and Heuristic Design

3.1 Benchmark Algorithm: A*

3.2 Baseline Heuristic: Manhattan Distance

3.3 Selected Uninformed Search Algorithm

3.4 Comparison Rationale

3.5 Heuristic Properties

3.6 New Heuristic Design

3.7 Expected Impact of the New Heuristic

4. Implementation Overview

4.1 Overall System Structure

4.2 Data Structures

4.3 Node Expansion Logic

4.4 Constraint Validation Logic

4.5 Implementation of A*

4.6 Implementation of the Uninformed Search Algorithm

4.7 Implementation of the New Heuristic

4.8 Code Snippet References

Listing 1: Placeholder for annotated code snippet with line numbers.

```
# Paste relevant code snippet here
# Make sure line numbers are visible in the PDF output
```

Listing 2: A* Search Algorithm

```
from queue import PriorityQueue
import math


def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path


def astar(graph, start, goal, heuristic):
    open_set = PriorityQueue()
    open_set.put((0.0, start))

    came_from = {}
    g_score = {start: 0.0}

    while not open_set.empty():
        _, current = open_set.get()

        if current == goal:
            return reconstruct_path(came_from, current), g_score[current]

        for neighbor, cost in graph.get(current, []):
            tentative = g_score[current] + cost

            if neighbor not in g_score or tentative < g_score[neighbor]:
                came_from[neighbor] = current
```

```
                g_score[neighbor] = tentative
                f_score = tentative + heuristic(neighbor, goal)
                open_set.put((f_score, neighbor))

    return None, float("inf")

# Build coordinates lookup (supports both dict and list landmarks format).
if isinstance(landmarks, dict):
    coords = {node_id: (data["x"], data["y"]) for node_id, data in landmarks.items()}
else:
    coords = {item["id"]: (item["longitude"], item["latitude"]) for item in landmarks}

def heuristic(node_a, node_b):
    xa, ya = coords[node_a]
    xb, yb = coords[node_b]
    return math.hypot(xa - xb, ya - yb)

# Build adjacency list with weighted edges.
graph_adj = {}
for p in paths:
    a, b = p["node_a"], p["node_b"]
    distance = p.get("distance", p.get("distance_km", 0))
    time = p.get("time", 0)
    risk = p.get("risk", 0)
    traffic = p.get("traffic_congestion", p.get("traffic", 0))
    is_directed = p.get("is_directed", False)
    if isinstance(is_directed, str):
        is_directed = is_directed.strip().lower() in ["true", "1", "yes", "directed"]

    omega = alpha * distance + beta * time
    cost = round((1 + risk + traffic) * omega, 2)
```

```
    graph_adj.setdefault(a, []).append((b, cost))
    if not is_directed:
        graph_adj.setdefault(b, []).append((a, cost))

start_node = 3
goal_node = 1
best_path, best_cost = astar(graph_adj, start_node, goal_node, heuristic)

if best_path is None:
    print(f"No route found from {start_node} to {goal_node}.")
else:
    print(f"A* path from {start_node} to {goal_node}: {best_path}")
    print(f"Total estimated cost: {best_cost:.2f}")
```

5. Results and Evaluations

5.1 Experimental Setup

5.2 Test Cases

Case	Start Node	Goal Node	Notes
1	3	1	Route from node 3 to 1
2	2	5	Route from node 2 to 5
3	1	4	Route from node 1 to 4

Table 1: Start-goal test cases used for evaluation.

5.3 Performance Metrics

5.4 Comparison of A* and Uninformed Search

Case	Algorithm	Path Found	Path Cost	Nodes Expanded	Runtime
1	A*	True	49.10	6	0.0591
1	Uninformed (Dijkstra)	True	0.00	18	0.0319

2	A*	True	11.67	3	0.0238
2	Uninformed (Dijkstra)	True	0.00	9	0.0172
3	A*	True	52.28	8	0.035
3	Uninformed (Dijkstra)	True	0.00	19	0.0279

Table 2: Performance comparison of A* and the selected uninformed search algorithm.

5.5 Evaluation of Intelligence Quality

5.6 Comparison of Heuristics

Case	Heuristic	Path Cost	Nodes Expanded	Runtime
1	Manhattan	0.00	18	0.052
1	New heuristic	0.00	18	0.031
2	Manhattan	0.00	9	0.023
2	New heuristic	0.00	9	0.02
3	Manhattan	0.00	19	0.032
3	New heuristic	0.00	19	0.03

Table 3: Comparison between Manhattan distance and the new heuristic.

6. Conclusion

Acknowledgement References

1. HanoiFastDelivery, *About HanoiFastDelivery*. Unpublished company profile for academic case study, 2026.
2. VietnamCredit, “Top 5 courier companies in Vietnam,” 2020. Accessed: 2026-04-14.
3. Google, *Google Maps*, 2026. Accessed: 2026-04-14.
4. United Nations Economic Commission for Europe, *Road Safety Performance Review: Viet Nam*, 2018. Accessed: 2026-04-14.
5. Deakin University, *SIT215: Computational Intelligence*. Unit materials, Deakin University, 2026. Accessed: 2026-04-14.
6. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson Education Limited, 2021. Global Edition. Accessed: 2026-03-14.
7. M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*. 6th ed. Wiley, 2014.
8. Author, *HanoiFastDelivery landmark and path dataset*. Unpublished project dataset, 2026. Based on 21 selected Hanoi landmarks and weighted road connections used in the implementation.